ON BOUNDED REFLECTION AS A MECHANISM FOR VERIFIED TRANSLATION EXTENSION

Isaac H. Lopez Díaz

University of Puerto Rico Río Piedras

2026

Professor

Tatiana Castro Velez, Faculty of Natural Sciences, Department of Computer Science

ABSTRACT

ON BOUNDED REFLECTION AS A MECHANISM FOR VERIFIED TRANSLATION EXTENSION

Isaac H. Lopez Díaz

Tatiana Castro Velez

DL frameworks all implement a source-to-source transformation — a staging boundary, in the sense of multi-stage programming — from an imperative host program to a captured computational graph. Under a categorical reading this boundary is a functor whose faithfulness is the framework's de facto correctness criterion, but no shipping framework establishes that faithfulness rigorously. We survey the design space, characterize the failures of functoriality in each system, and propose a typed reflective language in which the staging functor is constructed by elaboration and faithfulness is a theorem.

TABLE OF CONTENTS

CHAPTER 1

Introduction

Every modern deep-learning framework solves the same problem twice. First, it offers an eager imperative interface — write Python that builds tensors, call functions that perform operations, get answers back. Second, it offers a graph-capture mechanism — `jax.jit`, `torch.compile`, `@tf.function` — that takes the same Python and produces a computational graph suitable for compilation, autodifferentiation, and deployment. This second mechanism is, in the vocabulary of programming-languages research, a *source-to-source transformation*: a translation from one program text (a host-language fragment) into another (a graph IR), whose implicit correctness criterion is the preservation of the source program's semantics. The graph-capture mechanism is uniformly the part of the framework that practitioners learn to fear: silent retracing, mysteriously baked-in constants, host-side side effects that execute exactly once, dynamic shapes that defeat specialization. Frameworks document these hazards as engineering rules of thumb. They do not, as a rule, treat them as semantic facts.

This paper argues that they are semantic facts, and that programming-language theory has the tools to say what they are. A graph-capture mechanism is, in categorical terms, a functor from a category of source programs to a category whose morphisms are computational graphs. Two interpretations are then in play: the source-level meaning of a program (what it computes when run directly) and the graph-level meaning (what the captured graph denotes). The framework's implicit correctness criterion is that these two interpretations agree — that the capture functor is *faithful* with respect to denotational equivalence. The well-known hazards of `jit` and `compile` are precisely the failures of faithfulness: programs that have one meaning when run eagerly and another when captured.

The corresponding question for automatic differentiation has received careful treatment from the programming-languages community. Krawiec et al. (2022) gave a provably correct higher-order reverse-mode AD with a logical-relations correctness proof. Lee et al. (2020) addressed correctness in the presence of non-differentiable functions. Cruttwell et al. (2022) gave a categorical semantics

of gradient-based learning in terms of parametric lenses and reverse derivative categories. The same level of rigor has not been brought to bear on the staging primitive that precedes AD — the act of capturing the graph in the first place.

This paper surveys the design space of graph capture across six representative systems — JAX, PyTorch 2's TorchDynamo, TensorFlow AutoGraph, MLIR, Halide/TVM, and Dex — and reads each through a single categorical lens: what is the source category, what is the target category, what is the functor, and where does faithfulness fail? Read in this way, the systems differ less than their engineering vocabularies suggest. They are all instances of multi-stage programming in the sense of Taha and Sheard (2000), elaborated through a staging modality in the sense of Davies and Pfenning (2001). What distinguishes them is how aggressively they hide the staging boundary from the user, and how badly that hiding interacts with the host language's semantics.

The paper's contribution is twofold. First, it offers the categorical reading itself: a uniform vocabulary in which the gap between, say, JAX's tracer and TorchDynamo's bytecode rewriter is a question about which morphisms in **Set** lift along the capture functor. Second, drawing on the survey, it proposes a reflective core language with a typed staging modality, in which the capture functor is constructed by elaboration from the type system and faithfulness is a theorem provable by standard logical-relations techniques. The proposal is conceptual rather than implemented; the implementation is the subject of the author's ongoing thesis work on the reflective language Blue.

CHAPTER 2

Theoretical Foundations

2.1. Reflection, Staging, and Modal Type Theory

2.2. Categorical Semantics of Programs

2.3. Categorical Semantics of Differentiable Programs

2.4. Synthesis: Staging as a Functor over the DL Target

CHAPTER 3

Literature Review

3.1. Free Symmetric Monoidal Categories over a Primitive Signature

3.2. Quasi-Categorical Operator-Overloading Traces

3.3. AST-Rewriting Functors

3.4. Frame-Evaluation Reflective Functors

3.5. Verified Functors

3.6. Comparative Synthesis

Table 3.1 summarizes the six framework families along five axes: source category, target category, what the capture functor is, the known unsoundness patterns, and whether a correctness theorem is available.

System	**Source cat.**	**Target cat.**	**Functor**	**Known unsoundness**	**Thm.**
JAX					
TF AutoGraph					
TorchDynamo					
Halide / TVM					
Dex					
MLIR					

Table 3.1: Graph-capture mechanisms across six DL framework families, organized categorically.

CHAPTER 4

Analysis

4.1. Failure of Functoriality on Composition

4.2. Failure of Functoriality on Identity

4.3. Lack of Naturality

4.4. Forgetful Loss of Source Information

4.5. Impact

CHAPTER 5

New Proposal: Blue+

5.1. Davies–Pfenning Modal Core

5.2. Effect Typing on the Staged Fragment

5.3. Categorical Elaboration

5.4. The Faithfulness Theorem

Theorem 5.1 (Faithfulness, informal)**.** *For all well-typed Blue+ programs e, the diagram commutes: evaluating e in* **Set** *agrees with the universal interpretation of its elaboration in the free graph category.*

5.5. Conceptual Architecture

The compilation pipeline is sketched in Figure 5.1: typed core syntax elaborates into a generic CCC interface, which is then instantiated at a concrete category.

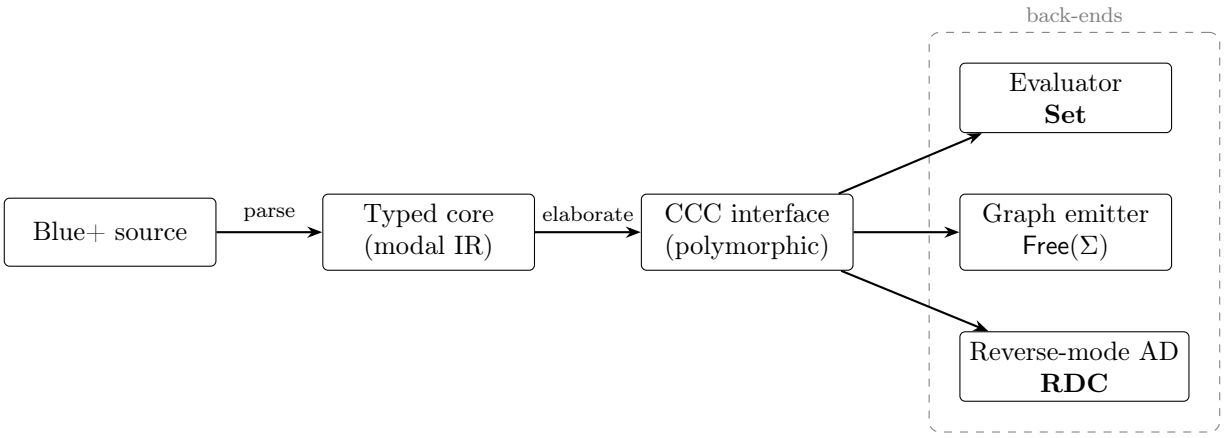

Figure 5.1: The compiling-to-categories template, with the staging modality made explicit and faithfulness a theorem rather than a hope.

5.6. Evaluation Methodology

CHAPTER 6

Conclusion

BIBLIOGRAPHY

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24), Volume 2*, pages 929–947. ACM, 2024. doi: 10.1145/3620665.3640366.

Brett Cannon, Dino Viehland Czajka, and Victor Stinner. PEP 523 – adding a frame evaluation API to CPython. https://peps.python.org/pep-0523/, 2016.

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*, pages 578–594. USENIX, 2018.

J. Robin B. Cockett, Geoffrey S. H. Cruttwell, Jonathan Gallagher, Jean-Simon Pacaud Lemay, Benjamin MacAdam, Gordon D. Plotkin, and Dorette Pronk. Reverse derivative categories. In *28th EACSL Annual Conference on Computer Science Logic (CSL 2020)*, volume 152 of *LIPIcs*, pages 18:1–18:16. Schloss Dagstuhl, 2020. doi: 10.4230/LIPIcs.CSL.2020.18.

Geoffrey S. H. Cruttwell, Bruno Gavranović, Neil Ghani, Paul Wilson, and Fabio Zanasi. Categorical foundations of gradient-based learning. In *Programming Languages and Systems: 31st European Symposium on Programming (ESOP 2022)*, volume 13240 of *Lecture Notes in Computer Science*, pages 1–28. Springer, 2022. doi: 10.1007/978-3-030-99336-8_1.

Rowan Davies and Frank Pfenning. A modal analysis of staged computation. *Journal of the ACM*, 48(3):555–604, 2001. doi: 10.1145/382780.382785.

Conal Elliott. Compiling to categories. *Proceedings of the ACM on Programming Languages*, 1 (ICFP):1–27, 2017. doi: 10.1145/3110271.

Brendan Fong, David I. Spivak, and Rémy Tuyéras. Backprop as functor: A compositional perspective on supervised learning. In *Proceedings of the 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '19)*, pages 1–13. IEEE, 2019. doi: 10.1109/LICS.2019.8785665.

Roy Frostig, Matthew J. Johnson, and Chris Leary. Compiling machine learning programs via high-level tracing. In *Conference on Systems and Machine Learning (SysML 2018)*, 2018.

Bruno Gavranović. *Fundamental Components of Deep Learning: A Category-Theoretic Approach*. PhD thesis, University of Strathclyde, 2024. PhD thesis.

Faustyna Krawiec, Simon Peyton Jones, Neel Krishnaswami, Tom Ellis, Richard A. Eisenberg, and Andrew Fitzgibbon. Provably correct, asymptotically efficient, higher-order reverse-mode automatic differentiation. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–30, 2022. doi: 10.1145/3498710.

Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain-specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 2–14. IEEE, 2021. doi: 10.1109/CGO51591.2021.9370308.

Wonyeol Lee, Hangyeol Yu, Xavier Rival, and Hongseok Yang. On correctness of automatic differentiation for non-differentiable functions. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 6719–6730. Curran Associates, 2020.

Damiano Mazza and Michele Pagani. Automatic differentiation in PCF. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–27, 2021. doi: 10.1145/3434309.

Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991. doi: 10.1016/0890-5401(91)90052-4.

Dan Moldovan, James M. Decker, Fei Wang, Andrew A. Johnson, Brian K. Lee, Zachary Nado, D. Sculley, Tiark Rompf, and Alexander B. Wiltschko. AutoGraph: Imperative-style coding with graph-based performance. In *Conference on Systems and Machine Learning (SysML 2019)*, 2019.

Adam Paszke, Daniel D. Johnson, David Duvenaud, Dimitrios Vytiniotis, Alexey Radul, Matthew J. Johnson, Jonathan Ragan-Kelley, and Dougal Maclaurin. Getting to the point: Index sets and parallelism-preserving autodiff for pointful array programming. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–29, 2021. doi: 10.1145/3473593.

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*, pages 519–530. ACM, 2013. doi: 10.1145/2491956.2462176.

Jared Roesch, Steven Lyubomirsky, Marisa Kirisame, Logan Weber, Josh Pollock, Luis Vega, Ziheng Jiang, Tianqi Chen, Thierry Moreau, and Zachary Tatlock. Relay: A high-level compiler for deep learning. In *arXiv preprint*, 2019.

Brian Cantwell Smith. Reflection and semantics in lisp. *Conference Record of POPL '84*, pages 23–35, 1984. doi: 10.1145/800017.800513.

Walid Taha and Tim Sheard. MetaML and multi-stage programming with explicit annotations. *Theoretical Computer Science*, 248(1–2):211–242, 2000. doi: 10.1016/S0304-3975(00)00053-0.